AI OPERATIONS (AIOps) · MODULE 4

Serving Models over REST APIs

Module 4 · Lecture (3 h) · Deck 1 of 4
From a trained file to a validated, observable, load-tested HTTP service

Third-year Data Science · CPU-only · all code runs on a laptop

Module 4 at a glance — one 3-hour lecture, four topics

Block	Topic	Minutes	Hands-on notebook
0	Intro: why serving is its own problem	5	—
1	REST API serving with FastAPI (this deck)	40	Lecture4a_REST_API_Serving
2	MLflow: pyfunc, Registry, `mlflow models serve`	40	Lecture4b_MLflow_Serving
—	Break	10	—
3	Ray Serve: replicas, canary, autoscaling	35	Lecture4c_Ray_Serve
4	Minikube: Deployments, Services, HPA, rollout	45	Lecture4d_Minikube_Serving
5	Wrap-up + 7-question quiz	5	Google Form

Core path of each notebook is done in class (~15–20 min); 'YOUR TURN' tasks and the remaining cells are take-home. BentoML is deliberately not covered.

Learning outcomes for this deck

Explain how online, batch and streaming serving differ, and when REST is the right choice

Build a FastAPI service with a typed schema, health probe and traceable responses

Verify that bad input yields 4xx (not 5xx) using a table of test cases

Measure latency, throughput and the saturation point with a load generator

Scale out with worker processes and reason about why it helps

Harden the client side with timeouts, retries and back-off

1

Why serving is its own problem

Training optimises accuracy once. Serving must be fast, available and safe — continuously.

Training vs serving: different jobs, different constraints

Training

- Runs for minutes–hours
- Throughput matters, latency does not
- Data fits in memory / cluster
- Failure = rerun
- Judged by: accuracy, cost per run

Serving

- Runs for months
- p95/p99 latency matters
- One request at a time (or small batch)
- Failure = user-visible outage
- Judged by: SLOs — latency, availability, error rate

Consequence

A model file is not a product.
You also need: an interface, validation, versioning, monitoring, scaling and rollback.

That is what Module 4 builds, layer by layer.

Three serving modes — choose by latency need, not by fashion

Mode	Trigger	Latency	Typical tool	Example
Online (request/response)	A user or service calls	ms – 100s of ms	REST / gRPC server	Churn score on a support call
Batch	A schedule	minutes – hours	Spark / `mlflow models predict`	Nightly scoring of all customers
Streaming	Events on a queue	100s of ms – s	Kafka consumer + model	Fraud flag per transaction

*This module: online serving. The *same* model artifact can usually serve all three modes — only the wrapper changes.*

Anatomy of an online prediction service



Every box can fail or be slow. The API layer is where most production bugs live: bad input, wrong column order, missing version info. Today we build the API layer by hand first, then let MLflow, Ray and Kubernetes generate or manage parts of it.

REST vs gRPC for model serving (syllabus: both)

	REST / JSON over HTTP	gRPC / Protobuf over HTTP/2
Payload	Human-readable JSON	Compact binary, typed schema (.proto)
Tooling	curl, browser, any language	Generated stubs, needs tooling
Speed	Slower (parse/serialise text)	Faster, streaming supported
Debuggability	Excellent	Harder (binary)
Best for	Public/simple APIs, prototypes, most tabular models	High-throughput internal calls, large tensors, streaming
Used today	FastAPI, MLflow, KServe V1, Ray Serve HTTP	KServe V2, Triton, Ray Serve gRPC (not covered)

The contract we will implement (KServe V1-style)

HTTP

```
POST /v1/models/churn:predict
Content-Type: application/json

{ "instances": [ {"tenure_months": 15, "monthly_charge": 26.23,
                  "support_calls": 0,  "late_payments": 0,
                  "data_usage_gb": 22.94, "contract_months": 12,
                  "num_products": 1, "age": 62} ] }

200 OK
{ "predictions":   [0],
  "probabilities": [0.0579],
  "model_version": "v1",          <- which model answered?
  "served_by":     "pod-7x2k" }   <- which replica answered?

GET /healthz  ->  {"status":"ok","model_version":"v1"}
```

Keep these names: the same URL shape reappears in Kubernetes (Lecture 4d). Traceability fields (`model_version`, `served_by`) make canary and incident analysis possible.

2

Build it: FastAPI service

Model loaded once · schema as code · health probe · traceable response

Setup — environment and the two model versions

```bash
# 1. environment (Python 3.10+)
python -m venv .venv && source .venv/bin/activate
pip install "fastapi>=0.110" "uvicorn>=0.30" scikit-learn pandas joblib requests matplotlib

# 2. train two versions of a churn classifier on synthetic data (a few seconds, CPU)
python train_models.py
#   v1: StandardScaler + LogisticRegression  -> data/model_v1.joblib  (2 KB)
#   v2: RandomForest (150 trees, depth 10)   -> data/model_v2.joblib  (8 MB)
#   held-out accuracy: v1 = 0.816   v2 = 0.840

# the joblib "bundle" stores the model AND its feature order:
#   {"model": ..., "feature_columns": [...8 names...], "version": "v2"}
```

Why save feature_columns with the model? Because the serving code must never guess column order — a classic silent-wrong-answer bug.

rest_app.py (1/3) — load once, define the schema

```python
MODEL_PATH = os.getenv("MODEL_PATH", "data/model_v1.joblib")
bundle = joblib.load(MODEL_PATH)      # loaded ONCE at startup, never per request
model, FEATURES, VERSION = bundle["model"], bundle["feature_columns"], bundle["version"]
POD = os.getenv("POD_NAME", socket.gethostname())

class Customer(BaseModel):                    # schema = contract with the client
    tenure_months: int = Field(ge=0, le=120)
    monthly_charge: float = Field(ge=0, le=500)
    support_calls: int = Field(ge=0)
    late_payments: int = Field(ge=0)
    data_usage_gb: float = Field(ge=0)
    contract_months: int = Field(ge=1, le=60)
    num_products: int = Field(ge=1, le=20)
    age: int = Field(ge=18, le=110)

class PredictRequest(BaseModel):
    instances: List[Customer] = Field(min_length=1, max_length=1000)
```

Each Field(ge=…, le=…) is a business assumption written as code. Invalid input is rejected before the model sees it.

rest_app.py (2/3) — middleware, health and metadata

```python
app = FastAPI(title="Churn model service")
STATS = {"requests": 0, "errors": 0}

@app.middleware("http")
async def timing(request: Request, call_next):
    t0 = time.perf_counter()
    resp = await call_next(request)
    resp.headers["X-Process-Time-ms"] = f"{(time.perf_counter()-t0)*1000:.2f}"
    STATS["requests"] += 1
    STATS["errors"] += int(resp.status_code >= 400)
    return resp

@app.get("/healthz")
def healthz():
    return {"status": "ok", "model_version": VERSION}

@app.get("/v1/models/churn")
def metadata():
    return {"name": "churn", "version": VERSION, "features": FEATURES, "served_by": POD}

@app.get("/stats")
def stats():
    return {**STATS, "served_by": POD, "model_version": VERSION}
```

A latency header and error counters are 'free' observability. Real systems export these as Prometheus metrics.

rest_app.py (3/3) — the predict endpoint

```python
@app.post("/v1/models/churn:predict")
def predict(req: PredictRequest):
    # plain `def` => runs in a thread pool: CPU-bound sklearn won't block the event loop
    try:
        X = pd.DataFrame([c.model_dump() for c in req.instances])[FEATURES]
        proba = model.predict_proba(X)[:, 1]
    except Exception as e:                    # never leak a stack trace to the client
        raise HTTPException(status_code=500, detail=f"inference failed: {type(e).__name__}")
    return {"predictions": [int(p >= 0.5) for p in proba],
            "probabilities": [round(float(p), 4) for p in proba],
            "model_version": VERSION, "served_by": POD}
```

Never leak a stack trace to the client: log it server-side, return a generic 500. Note `def` (not `async def`): see next slide.

`def` vs `async def` for model endpoints

async def (event loop)

- Runs ON the event loop thread
- Great for waiting: network, DB, other services
- A CPU-heavy predict() inside it BLOCKS every other request
- Use only with awaitable work

def (thread pool)

- FastAPI runs it in a worker thread
- Event loop stays free for health checks and I/O
- Right default for sklearn / numpy predict()
- Still limited by the GIL for pure-Python work → use multiple worker processes

Run it and call it

bash / python

```
# start (one worker)
MODEL_PATH=data/model_v1.joblib uvicorn rest_app:app --port 8000

# health + metadata
curl -s localhost:8000/healthz
curl -s localhost:8000/v1/models/churn

# predict (3 rows from the sample payload)
curl -s -X POST localhost:8000/v1/models/churn:predict \
    -H 'content-type: application/json' \
    -d @data/sample_payload.json     # {"instances":[{...},{...},{...}]}

# interactive docs generated from the schema:  http://localhost:8000/docs
# Python client
import requests
r = requests.post(URL, json={"instances": rows}, timeout=5); r.raise_for_status()
print(r.json()["probabilities"], r.headers["X-Process-Time-ms"])
```

3

Is it correct under bad input?

A serving API is a contract. Test the contract, not just the happy path.

Contract tests — what our API returned (measured)

Case	HTTP	Reason from the server
valid payload	200	—
negative tenure_months	422	Input should be greater than or equal to 0
missing field (age)	422	Field required
monthly_charge = "lots"	422	Input should be a valid number
empty instances list	422	List should have at least 1 item
wrong top-level key	422	Field required (`instances`)
age = 500	422	Input should be less than or equal to 110

Rule of thumb: 4xx = the client can fix it (don't retry); 5xx = our fault (client may retry). A 500 for bad input is a bug in the API.

Error-handling principles

Fail fast, fail clearly

Validate at the edge. Return a machine-readable reason. Never call the model on invalid data.

Don't leak

No stack traces, file paths or model internals in responses. Log them server-side with a request id.

Be explicit about retries

4xx → never retry. 5xx / timeout → retry with back-off and a cap. Idempotent predict calls make this safe.

Protect yourself

Cap batch size (we use 1000), body size and timeouts, or one client can take the service down.

4

Performance: latency, batching, saturation

Measure first. Intuition about performance is usually wrong.

Where does latency come from? (measured, 2-vCPU sandbox)

Batch size	Model only (ms)	HTTP total (ms)	Overhead (ms)	HTTP ms / row
1	1.7	7.4	5.7	7.38
10	2.0	7.6	5.6	0.76
100	2.2	9.5	7.3	0.095
500	2.0	15.5	13.5	0.031
1000	2.1	22.4	20.3	0.022

Your numbers will differ — the shape will not. A fixed per-request overhead dominates single rows (JSON, validation, TCP). Batching amortises it, but the first row waits for the last.

Load testing — what to measure and why

Closed-loop clients: N concurrent clients, each sends the next request when the previous returns

Report percentiles, not averages: p50 (typical), p95/p99 (what unlucky users see)

Little's Law: concurrency ≈ throughput × latency — when throughput flattens, extra clients only add queueing delay

Warm up first: first requests pay import/JIT/cache costs; discard them

Keep the generator off the server's CPU when you can (on a laptop you cannot — say so in your report)

load_test.py — the heart of the generator

```python
def run(url, fmt, conc, duration, rows, batch):
    body = payload(rows, fmt, batch)
    lat, errs, who, ver = [], 0, collections.Counter(), collections.Counter()
    end = time.time() + duration
    def worker():
        s = requests.Session(); out = []
        while time.time() < end:
            out.append(one(s, url, body))
        return out
    with cf.ThreadPoolExecutor(conc) as ex:
        for res in [f.result() for f in [ex.submit(worker) for _ in range(conc)]]:
            for ms, ok, w, v in res:
                lat.append(ms); errs += (not ok); who[w] += 1; ver[v] += 1
    lat.sort(); n = len(lat); q = lambda p: lat[min(n-1, int(p*n))]
    return dict(concurrency=conc, requests=n, rps=round(n/duration, 1), p50_ms=round(q(.5), 1),
                p95_ms=round(q(.95), 1), p99_ms=round(q(.99), 1), errors=errs,
                replicas=len(who), versions=dict(ver))
```

Usage: python load_test.py --url http://localhost:8000/v1/models/churn:predict --concurrency 1 2 4 8 16 --duration 4 --out sweep.json

Result: find the saturation knee (1 worker)

Concurrent clients	Throughput (req/s)	p50 (ms)	p95 (ms)	p99 (ms)	Errors
1	147	6.4	9.3	11.8	0
2	173	11.0	18.7	23.8	0
4	157	24.5	41.0	49.1	0
8	152	50.5	79.7	93.0	0
16	154	100.6	159.6	219.0	0

Throughput peaks near 2 clients (~170 req/s) and then stays flat while latency rises linearly: the single worker is saturated. Capacity of this service ≈ 150–170 req/s on this machine.

Scale out: more worker processes

```bash
# 2 worker processes (each loads its own copy of the model)
MODEL_PATH=data/model_v1.joblib uvicorn rest_app:app --port 8001 --workers 2

python load_test.py --url http://localhost:8001/v1/models/churn:predict \
      --concurrency 1 4 8 --duration 4

# measured (same machine, load generator competing for the 2 cores):
#   clients   1 worker     2 workers
#      4      152 req/s     276 req/s     (1.8x)
#      8      155 req/s     280 req/s     (1.8x)

# why it works: Python's GIL lets one process use ~1 core for pure-Python work;
# processes (not threads) add cores.  Memory cost: one model copy per worker.
```

Same idea, bigger scale: Ray Serve replicas (Deck 3) and Kubernetes pods (Deck 4).

A resilient client (Task A in the notebook)

```python
def safe_predict(rows, retries=3, base_url=BASE):
    delay = 0.2
    for attempt in range(retries + 1):
        try:
            r = requests.post(f"{base_url}/v1/models/churn:predict",
                              json={"instances": rows}, timeout=5)
        except requests.RequestException as e:          # network problem -> retry
            err = f"connection error: {type(e).__name__}"
        else:
            if r.status_code < 400: return r.json()
            if r.status_code < 500:                     # 4xx -> never retry
                raise ValueError(f"{r.status_code}: {r.json()['detail'][0]['msg']}")
            err = f"server error {r.status_code}"       # 5xx -> retry
        if attempt == retries: raise RuntimeError(f"gave up after {retries+1} attempts ({err})")
        time.sleep(delay); delay *= 2                   # exponential back-off
```

Production checklist for a REST model service

Health: /healthz for liveness AND readiness (Kubernetes uses it, Deck 4)

Versioning: model_version in every response; version in the URL or header

Limits: batch size, body size, timeout; reject early

Auth & TLS: API key / OAuth at the gateway; TLS terminated at the ingress

Observability: request id, latency histogram, error rate, input drift stats

Reproducibility: pinned dependencies, model baked into the image, same code path as training for features

Rollback: keep the previous version deployable at all times

HANDS-ON

Notebook 4a — REST API serving (core ≈ 15 min)

1. Run §0: it writes train_models.py, rest_app.py, load_test.py, m4util.py and trains v1/v2
2. §1–2: start the server, call it, then run the 7 contract-test cases — are all failures 422?
3. §3: reproduce the batch-size table; at what batch size does overhead per row stop improving?
4. §4: run the concurrency sweep; mark the knee on your plot
5. §5: restart with --workers 2 on port 8001 and compare
6. Take-home: YOUR TURN A (resilient client) and B (batch size under a latency budget)

Notebook: Lecture4a_REST_API_Serving_Exercise.ipynb

Common errors and fixes

Symptom	Likely cause	Fix
`Address already in use`	Old server still on the port	`pkill -f uvicorn` or choose another port
422 on a correct-looking payload	Wrong top-level key or missing field	Body must be {"instances": [{…all 8 features…}]}
`ModuleNotFoundError: rest_app`	Notebook cwd ≠ file location	Run `%pwd`; files are written to the notebook folder
Latency grows with clients, rps flat	Saturated worker (normal!)	More workers/replicas, or smaller work per request
Slow first request	Cold start / lazy imports	Warm up before measuring; use readiness probes in prod
Different results than in training	Column order / dtype mismatch	Always reindex to the saved feature_columns

Recap — REST serving

- A serving API = model + schema + health endpoint + traceability fields
- Validate at the edge: bad input is a 4xx, never a 500
- Latency = fixed overhead + compute; batching trades latency for throughput
- Load tests find the saturation knee; more processes/replicas move it right
- Clients need timeouts, retries (5xx only) and back-off

NEXT → Deck 2 — let MLflow generate the API and manage versions with its Registry